

writing-adapters

Writing Adapters

This guide walks through implementing a new platform adapter for pkg/userflow. Adapters are the extension point for adding support for new tools, platforms, or protocols.

When to Write a New Adapter

Write a new adapter when:

- You need to support a platform not covered by the built-in adapters.
- You want to use a different tool for an existing platform (e.g., Selenium instead of Playwright for browser testing).
- You need to customize the behavior of an existing adapter beyond what functional options allow.

The built-in adapters follow a CLI-based approach (shelling out to command-line tools). You may want to write adapters that use native Go libraries, gRPC, or HTTP APIs instead.

Step 1: Choose the Interface

Select the adapter interface that matches your platform:

Interface	File	When to Use
BrowserAdapter	adapter_browser.go	Web UI automation
MobileAdapter	adapter_mobile.go	Mobile device testing
DesktopAdapter	adapter_desktop.go	Desktop application testing
APIAdapter	adapter_api.go	REST API and WebSocket testing
BuildAdapter	adapter_build.go	Build, test, lint operations
ProcessAdapter	adapter_process.go	Process lifecycle management

Step 2: Implement the Interface

Create a new file following the naming convention `<tool>_<type>_adapter.go`. All adapters follow the same patterns.

Example: Selenium Browser Adapter

```
package userflow

import (
    "context"
    "time"
)

// SeleniumAdapter implements BrowserAdapter using Selenium
// WebDriver for browser automation.
type SeleniumAdapter struct {
    webDriverURL string
    sessionID    string
}

// Compile-time interface check.
var _ BrowserAdapter = (*SeleniumAdapter)(nil)

// NewSeleniumAdapter creates a SeleniumAdapter that connects
// to the given WebDriver URL.
func NewSeleniumAdapter(webDriverURL string) *SeleniumAdapter {
    return &SeleniumAdapter{
        webDriverURL: webDriverURL,
    }
}

func (a *SeleniumAdapter) Initialize(
    ctx context.Context, config BrowserConfig,
) error {
    // Create a WebDriver session with the configured browser
    // type, headless mode, and window size.
    // Store the session ID for subsequent calls.
    return nil
}

func (a *SeleniumAdapter) Navigate(
    ctx context.Context, url string,
) error {
    // POST /session/{id}/url with the target URL
    return nil
}

func (a *SeleniumAdapter) Click(
    ctx context.Context, selector string,
) error {
```

```

        // Find element by CSS selector, then click it
        return nil
    }

    func (a *SeleniumAdapter) Fill(
        ctx context.Context, selector, value string,
    ) error {
        // Find element, clear it, send keys
        return nil
    }

    // ... implement all remaining interface methods ...

    func (a *SeleniumAdapter) Available(
        ctx context.Context,
    ) bool {
        // Check if the WebDriver endpoint is reachable
        return false
    }

```

Key Implementation Requirements

1. **All methods must accept `context.Context`** as the first parameter (except `IsRunning` and `Stop` on `ProcessAdapter`). Respect context cancellation.
2. **`Available()` must be non-destructive.** It should check tool availability without side effects (no processes started, no files created).
3. **`close()` must be idempotent.** Calling it multiple times should not produce errors.
4. **Error wrapping:** Wrap errors with `fmt.Errorf("operation: %w", err)` to provide context.
5. **Compile-time interface check:** Add a blank assignment to verify interface satisfaction:

```
var _ BrowserAdapter = (*MyAdapter)(nil)
```

Step 3: Handle Tool-Specific Output

Many adapters need to parse tool output. Follow these patterns from the built-in adapters:

JSON Output Parsing (Go, Cargo, ESLint)

```
type toolEvent struct {
    Action string `json:"action"`
}
```

```

    Name    string `json:"name"`
}

scanner := bufio.NewScanner(strings.NewReader(output))
for scanner.Scan() {
    var ev toolEvent
    if err := json.Unmarshal([]byte(scanner.Text()), &ev); err !=
        nil {
        continue // skip non-JSON lines
    }
    // process event
}

```

JUnit XML Parsing (Gradle, npm)

Use the built-in ParseJUnitXML and JUnitToTestResult:

```

data, err := os.ReadFile(xmlPath)
if err != nil {
    return nil, err
}
suites, err := ParseJUnitXML(data)
if err != nil {
    return nil, err
}
result := JUnitToTestResult(suites, elapsed, output)

```

Plain Text Parsing (ADB instrument)

For tools that output plain text, use string matching:

```

lines := strings.Split(output, "\n")
for _, line := range lines {
    if strings.HasPrefix(line, "OK (") {
        var n int
        fmt.Sscanf(line, "OK (%d tests)", &n)
    }
}

```

Step 4: Command Execution

For CLI-based adapters, use os/exec with context:

```

func (a *MyAdapter) runTool(
    ctx context.Context, args ...string,
) (string, error) {
    cmd := exec.CommandContext(ctx, "toolname", args...)
    cmd.Dir = a.projectRoot

    out, err := cmd.CombinedOutput()

```

```

    if err != nil {
        return string(out), fmt.Errorf(
            "toolname %v: %w", args, err,
        )
    }
    return string(out), nil
}

```

For adapters that need persistent connections (WebDriver, CDP), use net/http:

```

func (a *MyAdapter) wdPost(
    ctx context.Context, path, body string,
) ([]byte, error) {
    req, err := http.NewRequestWithContext(
        ctx, http.MethodPost,
        a.baseURL+path,
        bytes.NewBufferString(body),
    )
    if err != nil {
        return nil, err
    }
    req.Header.Set("Content-Type", "application/json")

    resp, err := a.httpClient.Do(req)
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()

    data, err := io.ReadAll(resp.Body)
    if err != nil {
        return nil, err
    }
    if resp.StatusCode >= 400 {
        return data, fmt.Errorf("HTTP %d: %s", resp.StatusCode,
            data)
    }
    return data, nil
}

```

Step 5: Write Tests

Follow the naming convention `<tool>_<type>_adapter_test.go`. Test against the interface:

```

func TestMyAdapter_Navigate(t *testing.T) {
    adapter := NewMyAdapter("http://localhost:4444")

    ctx := context.Background()
    if !adapter.Available(ctx) {

```

```

        t.Skip("tool not available")
    }

    err := adapter.Initialize(ctx, BrowserConfig{
        Headless: true,
        WindowSize: [2]int{1024, 768},
    })
    require.NoError(t, err)
    defer adapter.Close(ctx)

    err = adapter.Navigate(ctx, "http://example.com")
    assert.NoError(t, err)
}

```

Testing Patterns from the Codebase

- Use `t.Skip` when the tool is not available (allows tests to pass in CI without the tool installed).
- Test the `Available()` method explicitly.
- Test error conditions (invalid paths, unreachable endpoints).
- For build adapters, create temporary project directories with minimal config files.

Step 6: Integrate with Challenge Templates

The new adapter can be used with existing challenge templates immediately, since templates depend on interfaces, not concrete types:

```

adapter := NewMySeleniumAdapter("http://localhost:4444")

challenge := NewBrowserFlowChallenge(
    "CH-BROWSER-SELENIUM-001",
    "Login via Selenium",
    "Test login with Selenium adapter",
    nil,
    adapter, // works because it implements BrowserAdapter
    flow,
)

```

Adapter Checklist

Before finalizing a new adapter, verify:

☐

Implements all interface methods.

☐

Has a compile-time interface check (`var _ Interface = (*Adapter)(nil)`).

☐

Has a constructor function (`NewXxxAdapter`).

☐

Respects `context.Context` cancellation.

☐

Wraps errors with descriptive messages.

☐

`Available()` is non-destructive.

☐

`Close()` is idempotent.

☐

Has unit tests with `t.Skip` for missing tools.

☐

Follows the naming convention: `<tool>_<type>_adapter.go`.

☐

Has godoc comments on all exported types and functions.